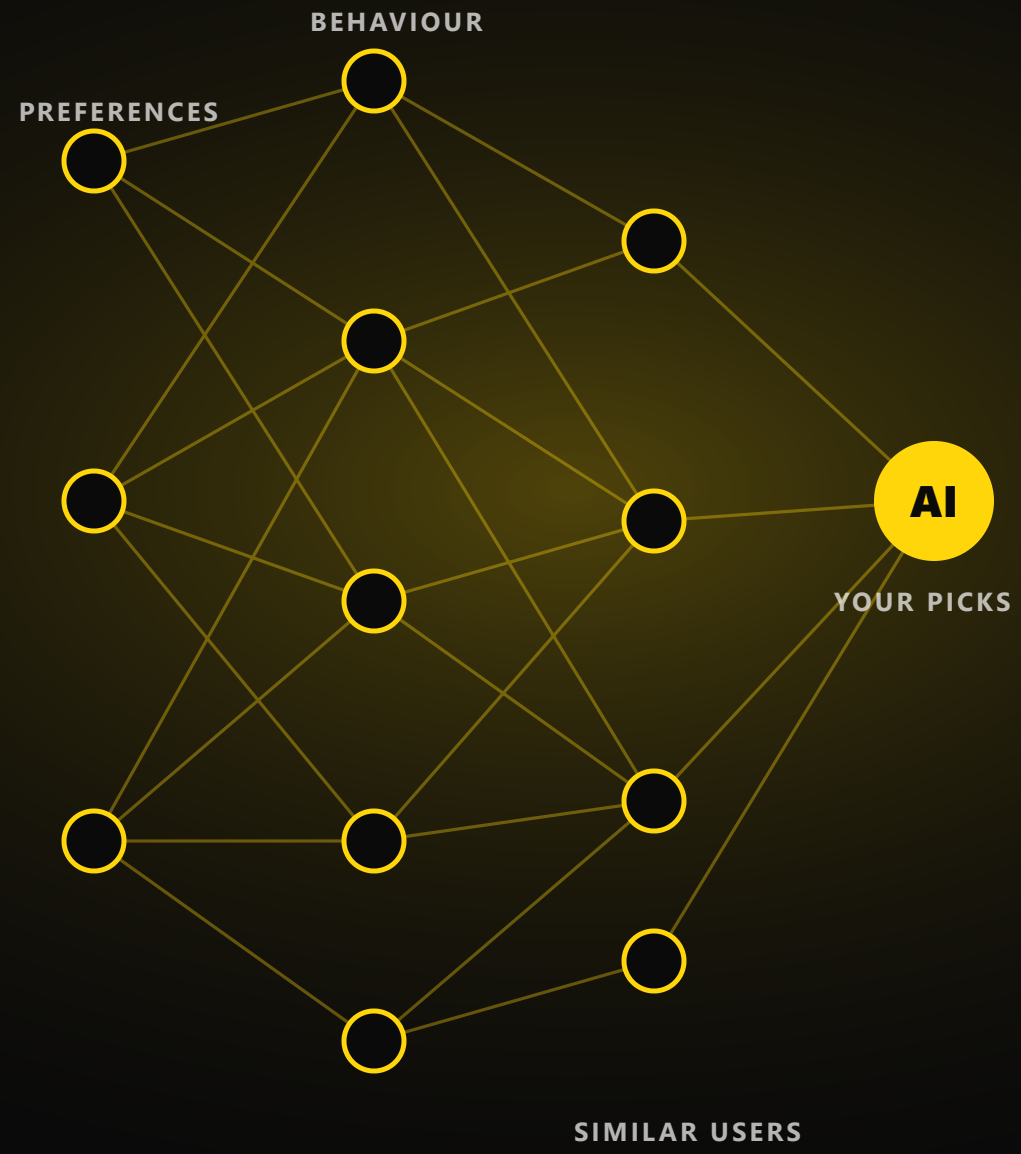
JAVA WEB BASED PROJECT · RUBRIC B

AI-Based Recommendation System

Pulse suggests products and content from each user's preferences and behaviour, with an admin console to tune and measure the AI.

Team Leader Vedang Goswami

Members Ankur Kushwaha · Nitin Raghuvanshi



OUR TEAM

Who built Pulse

V

Vedang Goswami

Team Leader

- Project planning, architecture and task split
- Recommendation engine and parallel scorers
- Servlets, filters, sessions and security
- Final integration, testing and presentation

A

Ankur Kushwaha

Team Member

- Database schema and JDBC data-access layer
- Transactions and catalogue loading (251 products)
- Analytics service and reviews backend
- JUnit integration tests

N

Nitin Raghuvanshi

Team Member

- Front-end UI/UX and black-white-yellow design system
- Discover screen, product dialog, admin dashboards
- Product research, pictures and image credits
- Documentation and diagrams

WHY THIS PROJECT

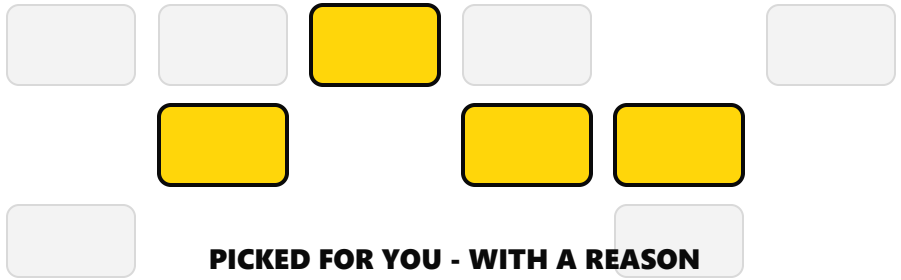
Problem versus solution

THE PROBLEM



- Hundreds of products - users scroll, get lost and leave
- Same generic list for everybody
- New users get nothing useful (cold start)
- Admins cannot see which suggestions actually work

OUR SOLUTION



- Hybrid AI: your interests + similar users + trends
- "Why this?" shown on every card
- Interest studio: tell the AI, feed updates live
- Admin analytics and one-click tuning suggestions

REQUIREMENTS

Two user types, four core functions

Admin

FUNCTION	INPUT → OUTPUT
System settings weights, min score, list size, diversity	Settings → confirmation of update
Effectiveness analysis CTR, trends, per-signal, per-category	Recommendation data → reports + suggestions
User management profiles, roles, activate / delete	Admin action → updated user

User

FUNCTION	INPUT → OUTPUT
Personalised recommendations with match % and explanation	Behaviour + preferences → suggested products / content
Preference management interest levels, topics, name	Preferences → confirmation of update
Reviews and feedback stars + text, also trains the AI	Review → shown to everyone

2 dashboards

251 real products

14 categories

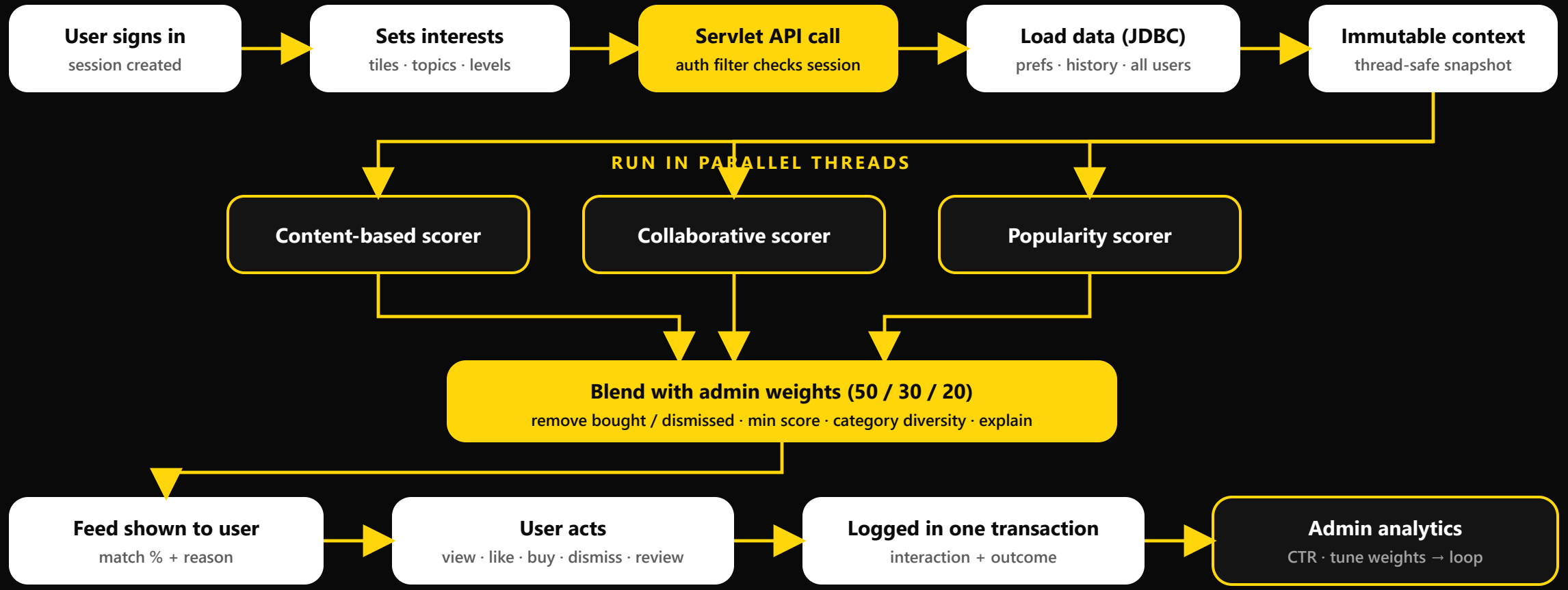
3 AI signals

Admin dashboard: **System Settings + Recommendation Analysis**. User dashboard: **Recommendation List + Preference Settings** - combined into one live "Discover" screen.

SYSTEM WORKFLOW

How a recommendation is made

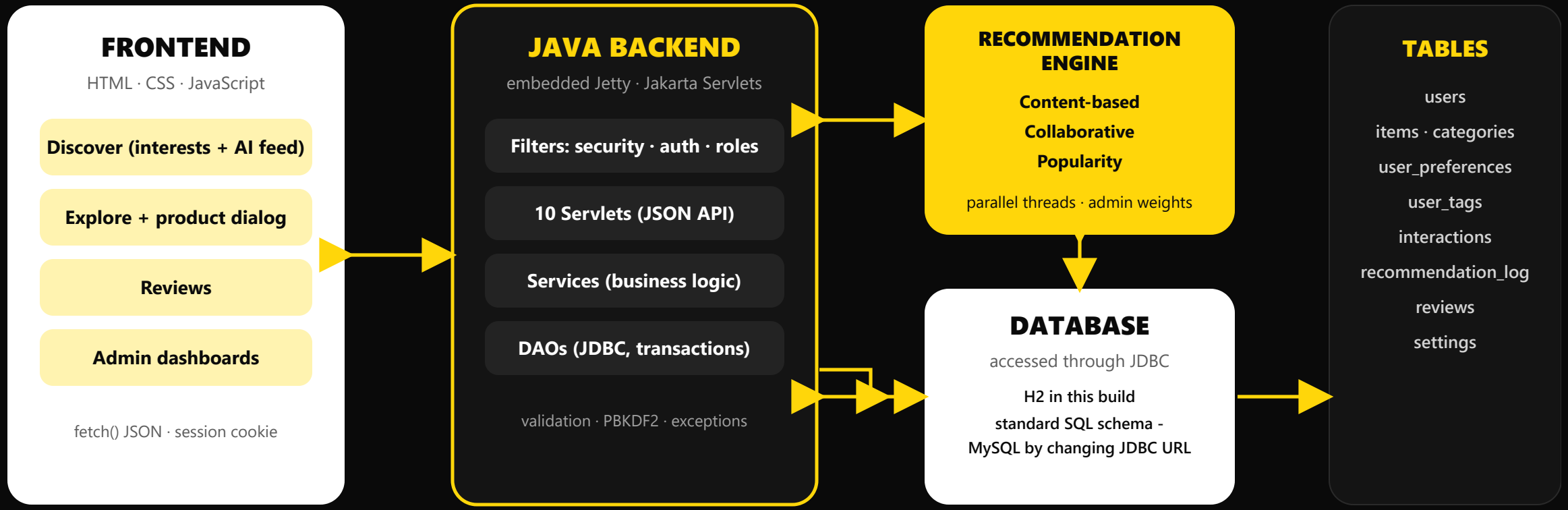
DIAGRAM



ARCHITECTURE

Frontend → Java backend → engine → database

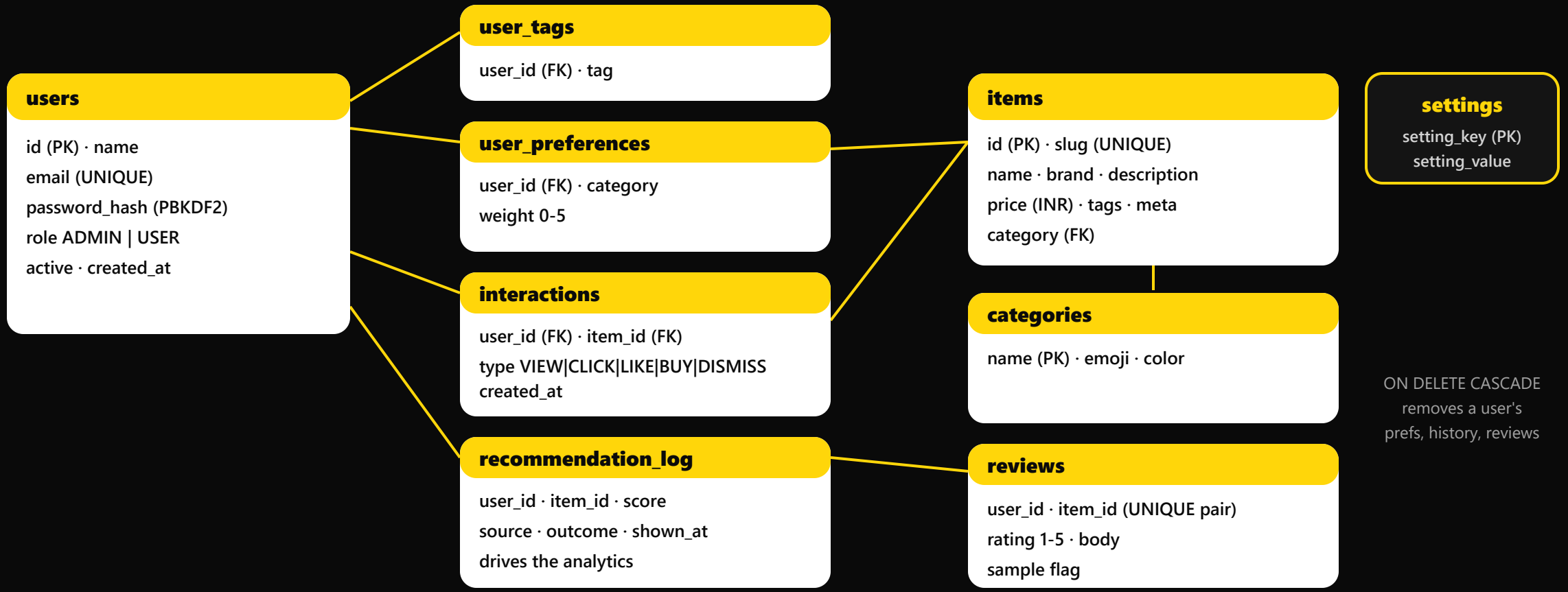
DIAGRAM



DATABASE DESIGN

Relational schema (ER diagram)

DIAGRAM



THE AI

A hybrid recommender with three signals

$$\text{score(item)} = w_1 \cdot \text{Content} + w_2 \cdot \text{Collaborative} + w_3 \cdot \text{Popularity}$$

weights are set by the admin, normalised to 100%, and applied immediately

1 · Content-based

Builds a profile of your categories and topics from what you chose **and** what you did (recency-decayed). Scores items sharing those features.

default weight 50%

2 · Collaborative

Finds the 10 users most similar to you (cosine similarity) and recommends what they loved that you have not seen.

default weight 30%

3 · Popularity

What the whole community engages with right now. Also solves the cold-start problem for brand-new users.

default weight 20%

After blending: remove items already bought or dismissed · drop low scores · limit items per category (diversity) · attach a plain-English reason.

Runs in parallel: the three scorers execute on a thread pool (CompletableFuture), so a recommendation takes milliseconds.

CONTENT-BASED FILTERING

How your interests become a score

ILLUSTRATIVE EXAMPLE · SAME RULES AS CONTENTBASEDSCORER

User profile

built from the Interest Studio + past clicks

Category **Books** ★★★★★

Category **Movies** ★★★★☆

#sci-fi **#classic**

Each feature gets a weight: category level ÷ 5, topics +0.6, recent behaviour adds more.

Dune
Books · Frank Herbert

Books ✓ **#sci-fi ✓** **#classic ✓** #epic

100%
3 shared features

Forrest Gump
Movies · R. Zemeckis

Movies ✓ **#classic ✓** #drama

62%
2 shared features

Echo Dot
Wearables & Smart Home

#smart-speaker #alexa #audio

≈0%
no shared features

Learning from other people

DIAGRAM



Popularity adds up engagement from every user, giving recent events more weight (half-life 14 days, admin-tunable). New users with no history still get sensible suggestions.

SAMPLE RECOMMENDATION OUTPUT

Change the preferences → the picks change

REAL OUTPUT CAPTURED FROM THE RUNNING APPLICATION

Profile A

Books ★5 · Movies ★4 | #sci-fi | #classic

TOP PICKS	MATCH	WHY
The 7 Habits of Highly Effective People Books	**74%**	Matches your interest in classic
The Hitchhiker's Guide to the Galaxy Books	**63%**	sci-fi & classic
Forrest Gump Movies	**62%**	Matches classic
The Godfather Movies	**58%**	Matches classic
Tapo C200 Security Camera Smart Home	**57%**	Loved by similar users

Profile B

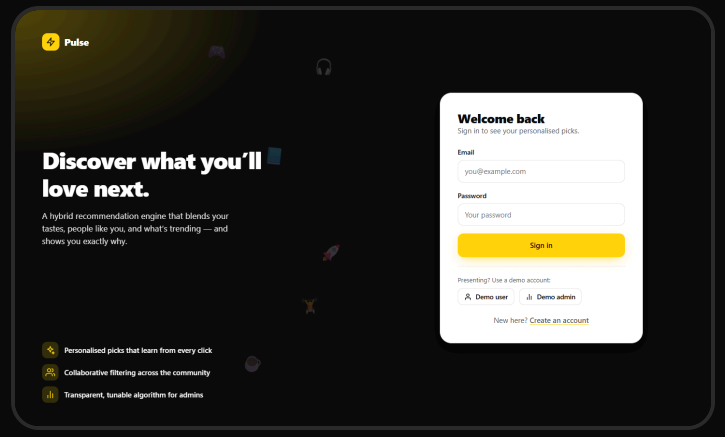
Fitness ★5 · Wearables ★4 | #running | #fitness

TOP PICKS	MATCH	WHY
Tapo C200 Security Camera Smart Home	**70%**	Loved by similar users
Mi 360° Home Security Camera 2K Smart Home	**53%**	Because you enjoy Wearables
Speedgoat 5 Trail Shoes Fitness	**53%**	Matches running
Forerunner 55 Fitness	**52%**	running & fitness
GRID Foam Roller Fitness	**51%**	Matches fitness

Same user account, same engine - only the interests changed. Books and movies dominate profile A; running shoes, a sports watch and a foam roller appear for profile B.

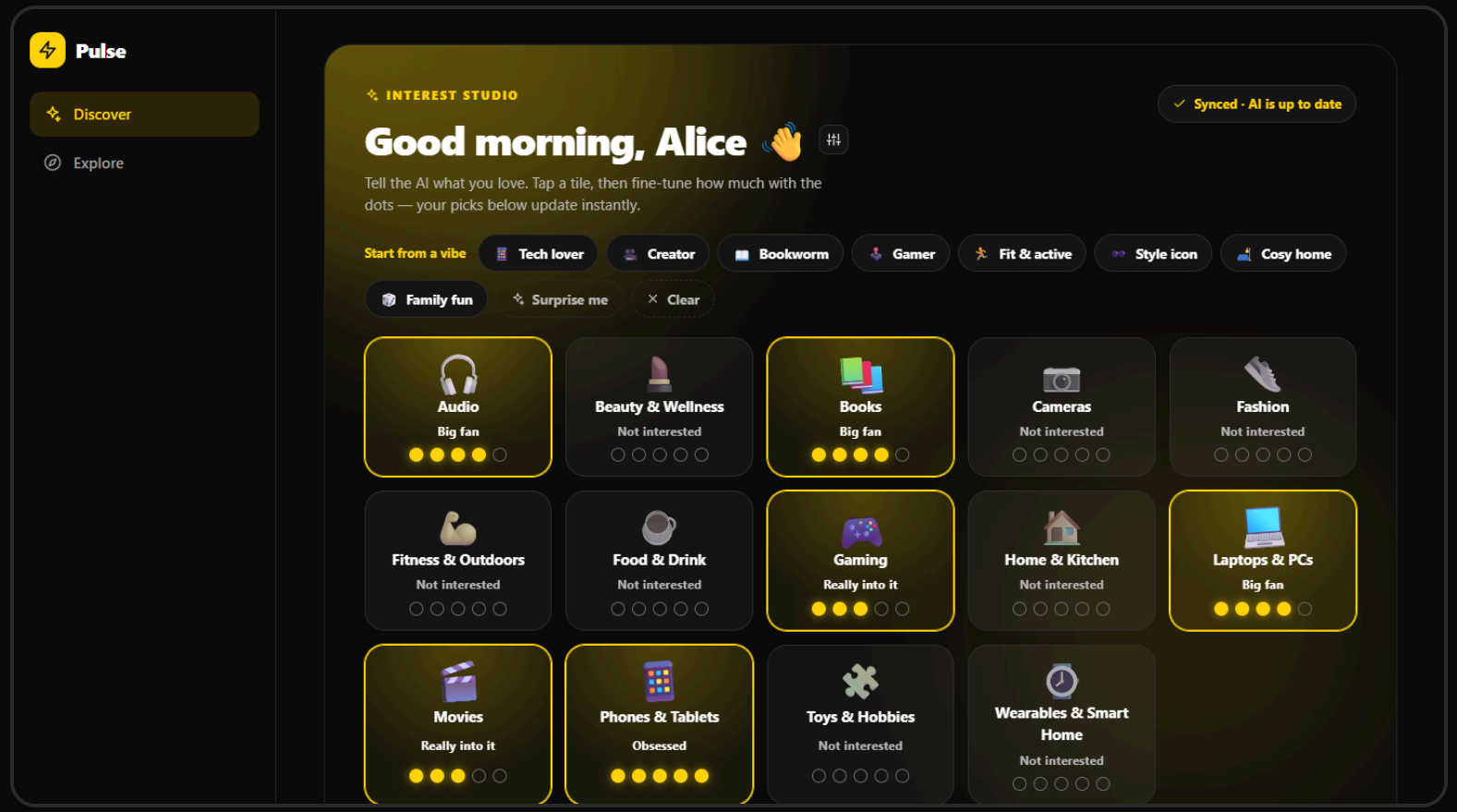
WORKING APPLICATION

Sign in and tell the AI what you love



Screenshot · Login page with demo accounts for presenting

- Secure sessions, PBKDF2 passwords
- Register with starting interests

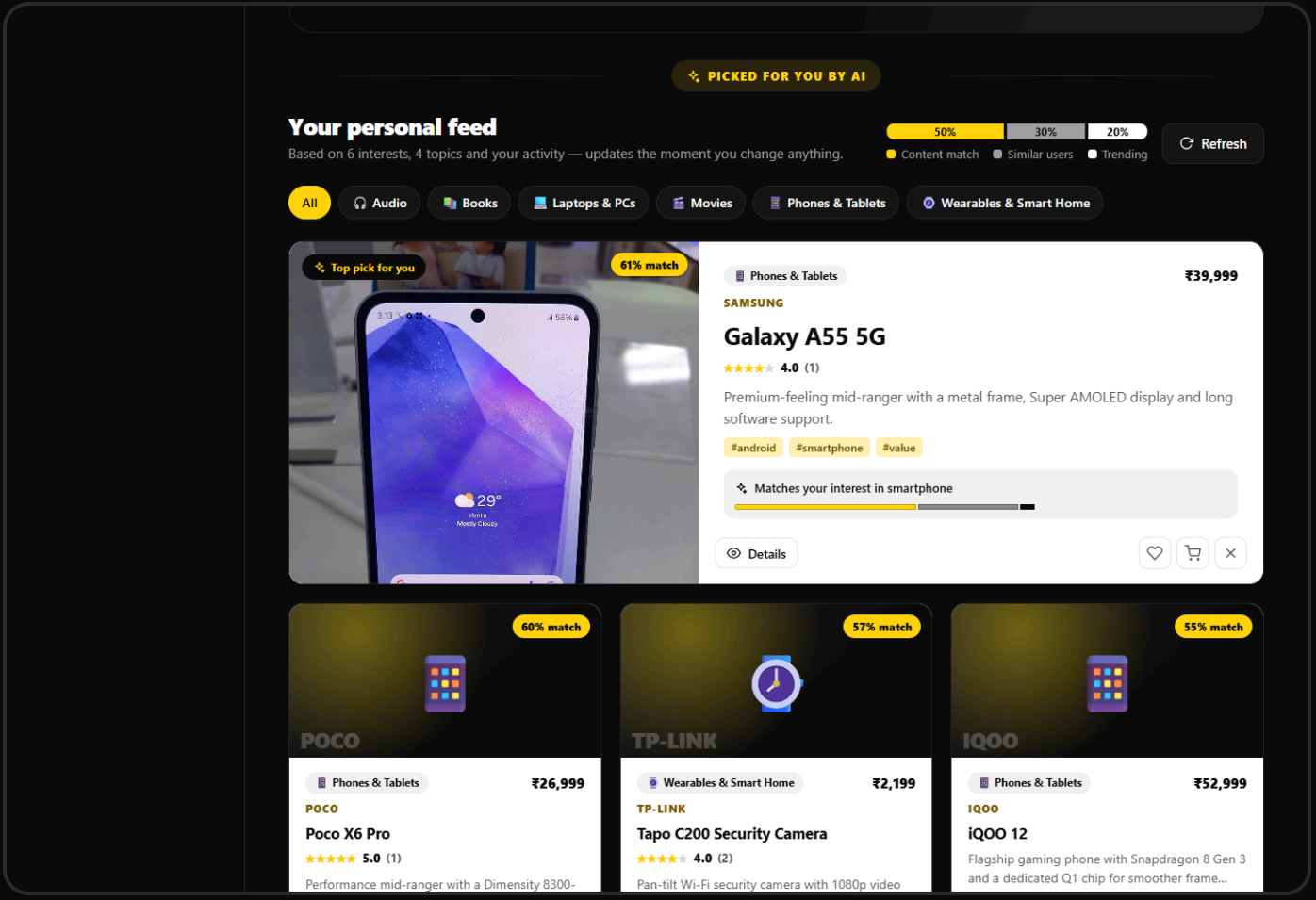


Screenshot · Discover - the Interest Studio (14 categories, levels, vibes, taste DNA)

- Tap a tile, fine-tune with dots, or start from a vibe
- Changes autosave in about a second

USER DASHBOARD

Personal feed with an explanation on every card



Screenshot · The AI feed under the studio - the preferences above are what produced these picks

What the user sees

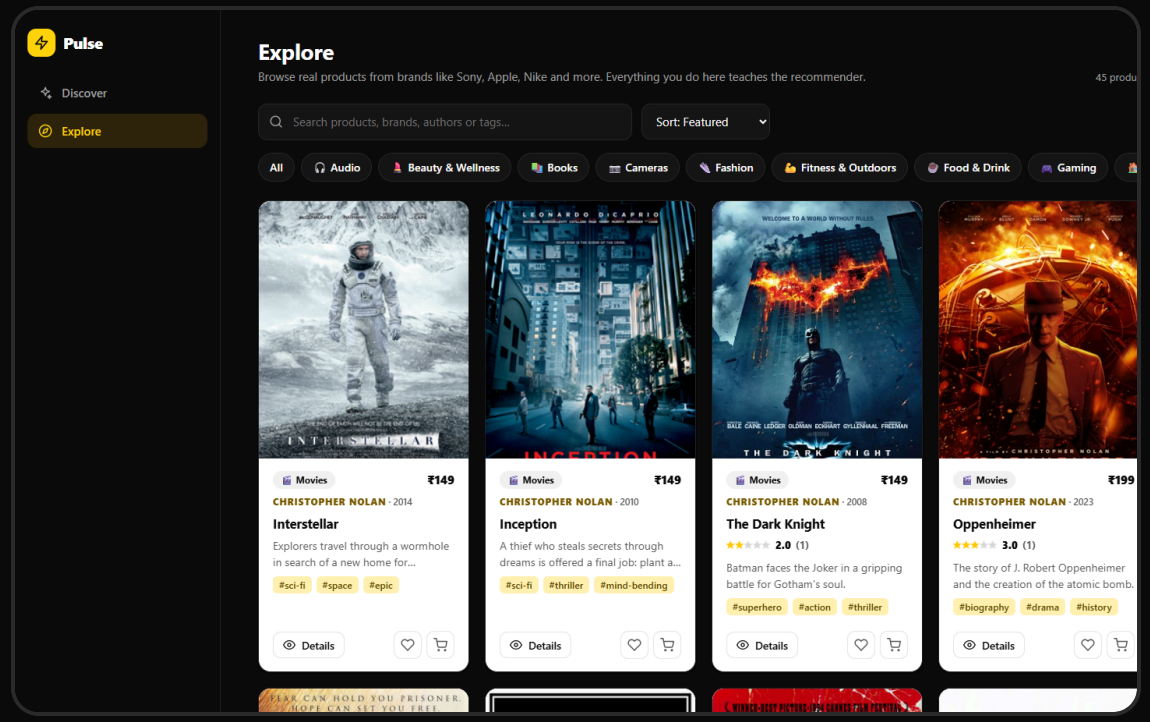
- **Top pick** as a large featured card
- **Match %** from the blended score
- **Reason**: "Matches your interest in smartphone"
- **Signal bar**: content / similar users / trending

What the user can do

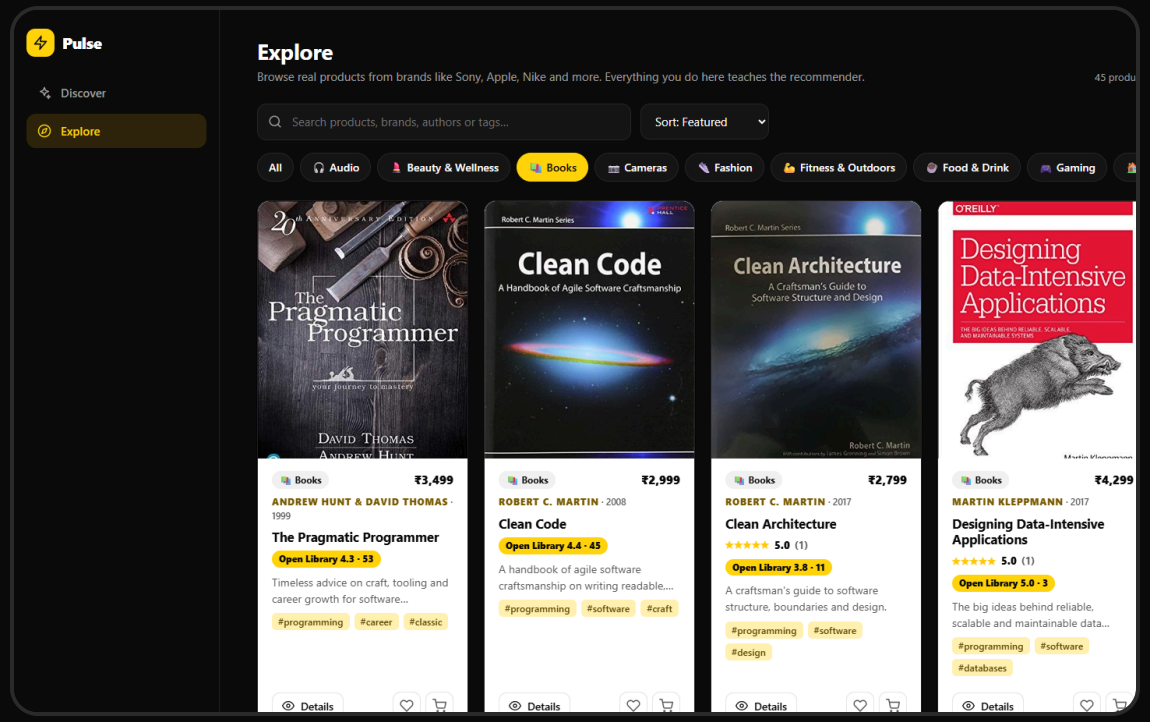
- ♥ like · 🛒 buy · ✕ not interested
- Open details and write a review
- Every action trains the next list

REAL CATALOGUE

251 real products, books and films



Screenshot · Explore - real film posters

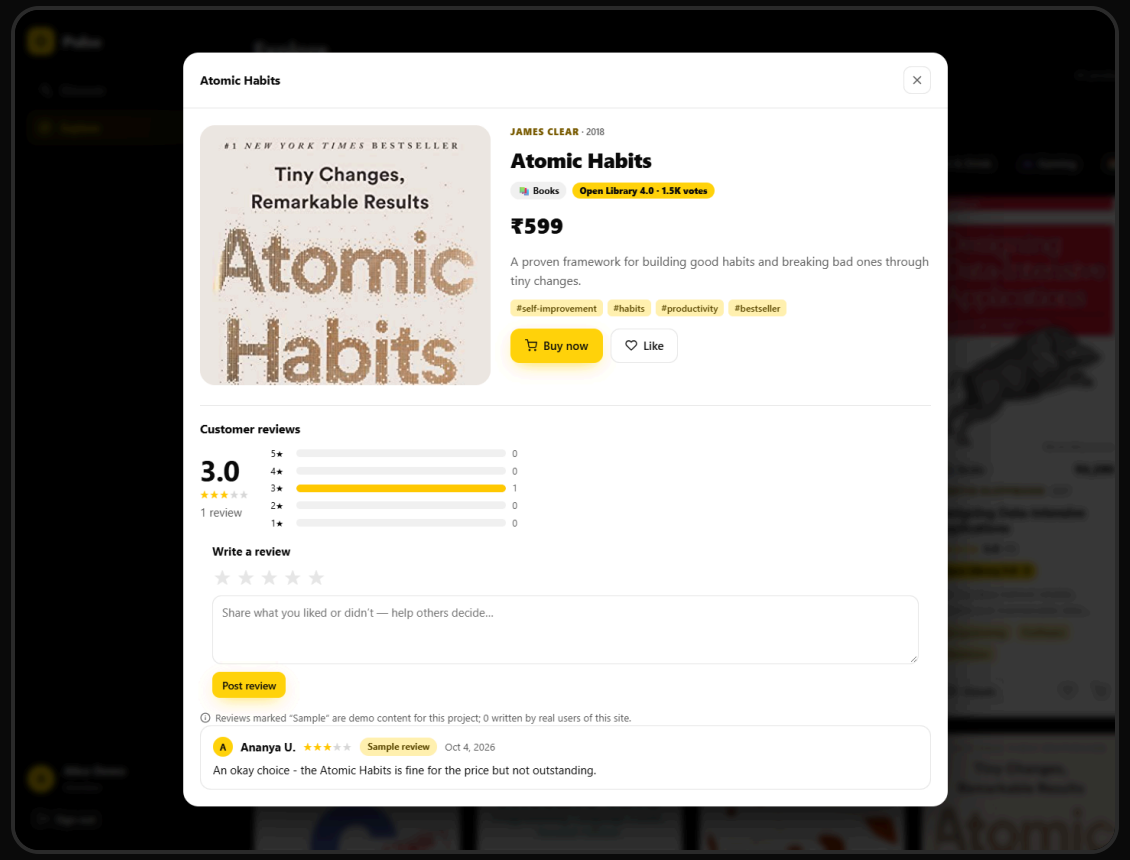


Screenshot · Explore - real book covers with public reader ratings

14 categories - phones, laptops, audio, cameras, wearables, gaming, books, movies, fashion, home, fitness, food, beauty, toys

Search, filter, sort by price or brand. Pictures from Open Library and Wikipedia / Wikimedia, credited in the repository.

Reviews that also teach the AI



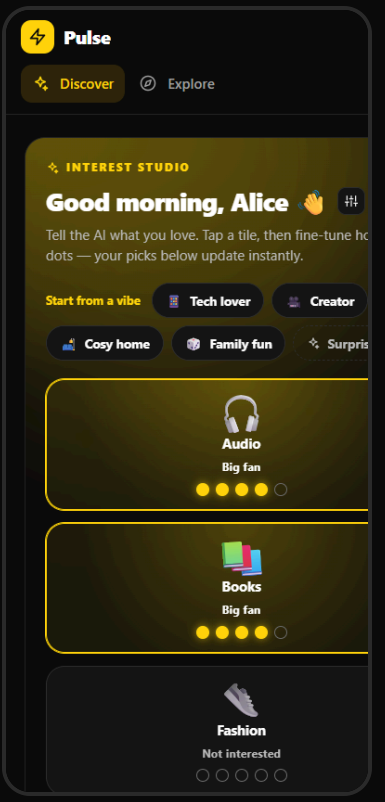
Screenshot · Product dialog - cover, rating, star histogram, write-a-review form

Customer feedback

- Users post star reviews
- A 4-5★ review = a **like** for the recommender; 1-2★ = **not interested**
- Demo reviews are clearly labelled **Sample review**
- Public ratings (Open Library) shown on books

Responsive

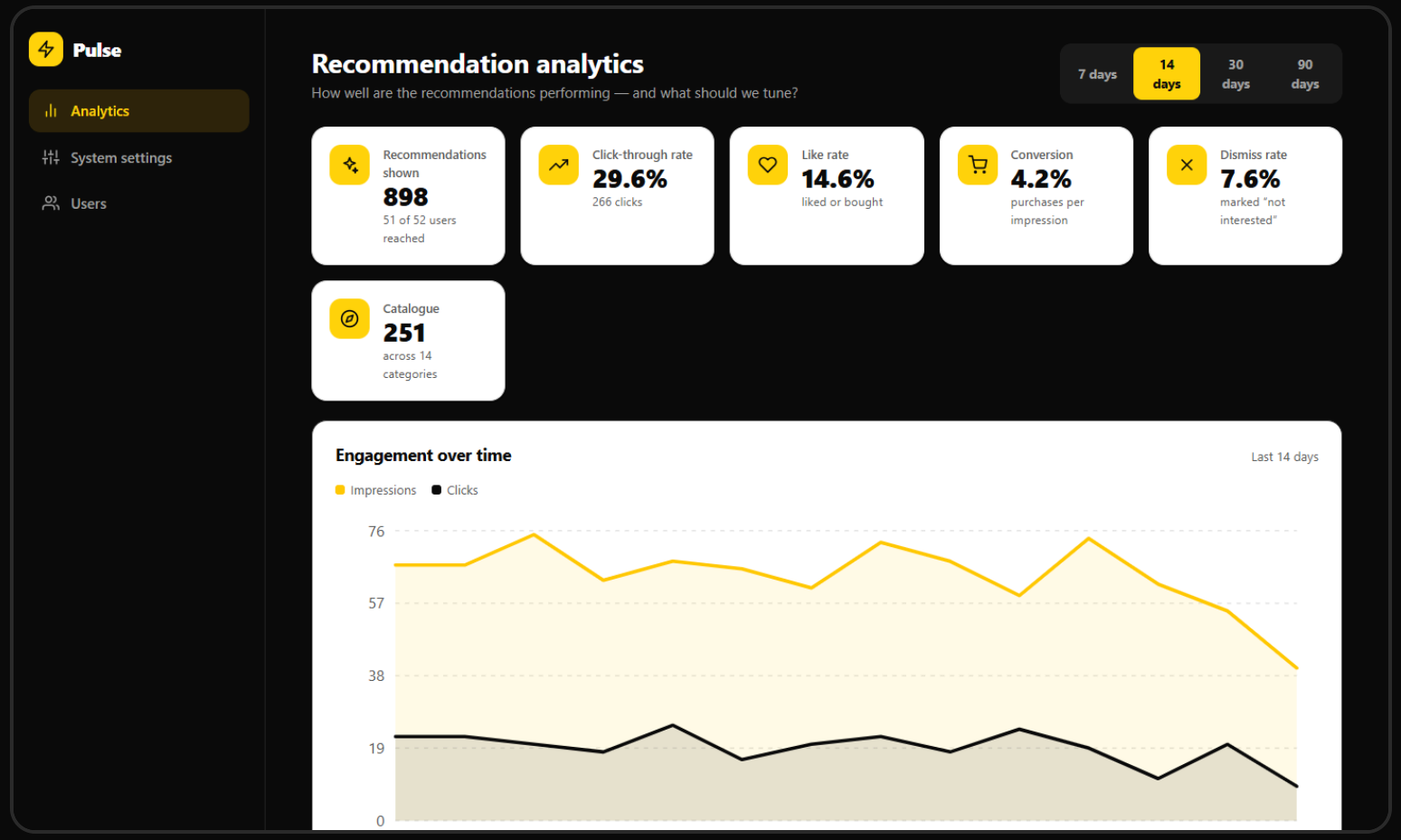
The same UI works on phones - tiles become two columns.



Screenshot · Mobile

ADMIN DASHBOARD · RECOMMENDATION ANALYSIS

Which recommendations actually work?



Screenshot · Admin analytics with live charts

Reports

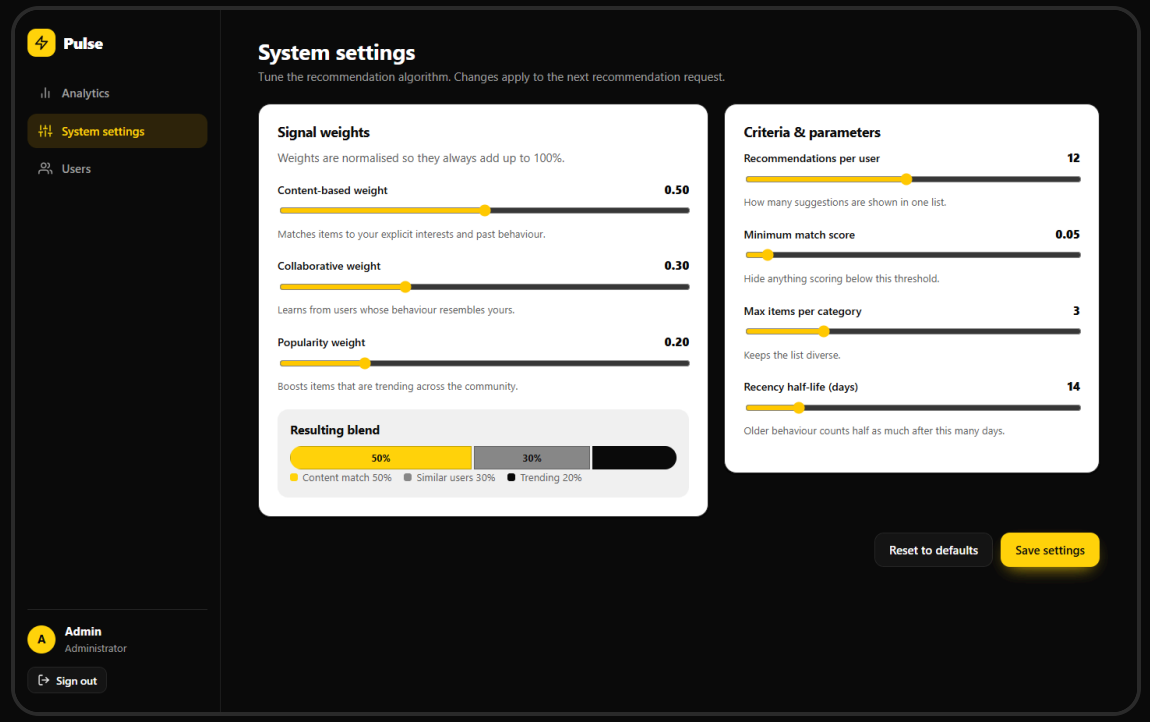
- KPIs: shown, CTR, like, conversion, dismiss
- 14 / 30 / 90-day trend chart
- Click-through per AI signal and per category
- Top performing items

Smart suggestions

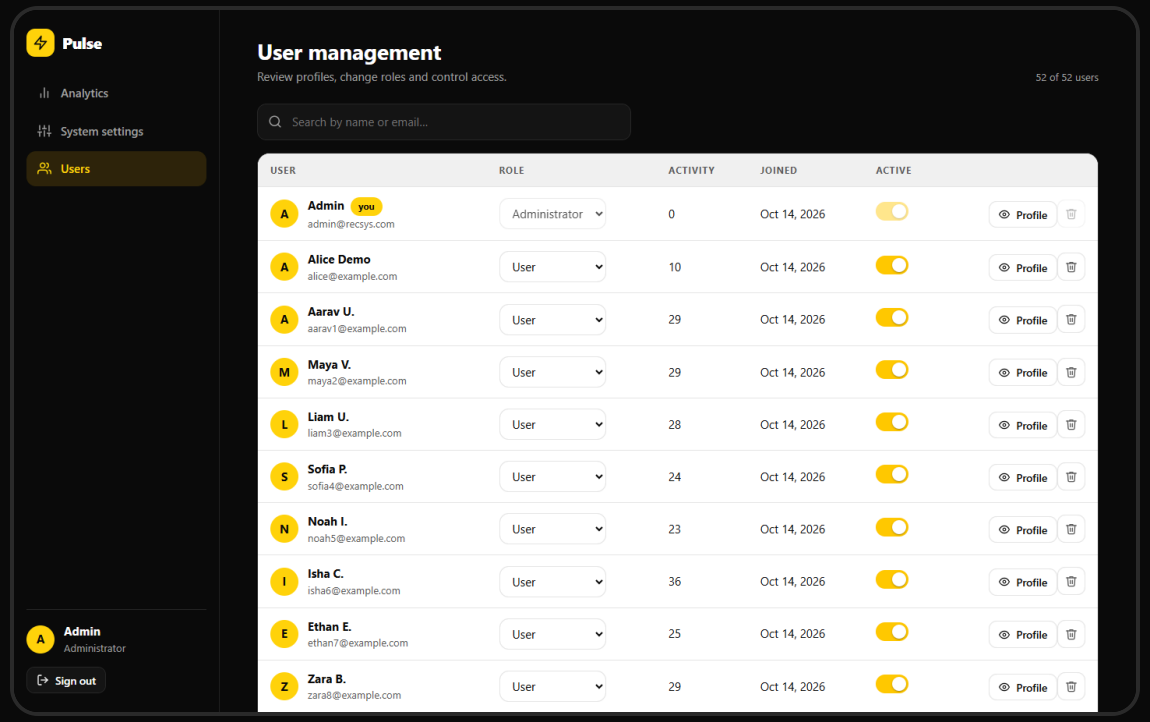
Rule-based advice such as "shift weight toward collaborative signals" with an **Apply suggestion** button that updates the settings.

ADMIN DASHBOARD · SYSTEM SETTINGS & USERS

Tune the algorithm, manage people



Screenshot · System settings - weights with live blend preview, criteria, save / reset



Screenshot · User management - roles, activate, profile, delete

Validated & atomic: every setting is checked on the server and saved in one transaction.

Safe: the last administrator cannot be deleted or demoted; admins cannot remove themselves.

CORE JAVA CONCEPTS

OOP · Collections · Exceptions · Threads

Object-oriented design

- **Interface + polymorphism:** `Scorer` with three implementations (Strategy pattern)
- **Inheritance:** `ApiServlet` base class for 10 servlets
- **Encapsulation:** records and a validating `RecommendationSettings`
- **Enums with behaviour:** `InteractionType`, `RecOutcome`

Collections

- `HashMap / LinkedHashMap` for profiles and vectors
- `PriorityQueue` - bounded top-10 neighbours
- `Set / LinkedHashSet` for exclusions and tags
- `TreeMap`, streams and `Comparator` chains
- `ConcurrentHashMap` for login throttling

Exception handling

- Custom hierarchy: Validation · Authentication · Forbidden · NotFound · Conflict · DataAccess
- One central handler turns them into JSON errors with the right HTTP status
- try-with-resources and transaction rollback

Threads & concurrency

- `ExecutorService` + `CompletableFuture`: scorers run in parallel with a timeout
- `ScheduledExecutorService` housekeeping job
- Immutable context object, `volatile` settings cache

DATABASE & WEB INTEGRATION

JDBC transactions, Servlets and sessions

JDBC

- 9 tables with PK, FK, CHECK, UNIQUE, indexes, ON DELETE CASCADE
- CRUD through `PreparedStatement` (no SQL injection)
- Connection pool and batch inserts
- **Transactions** (commit / rollback): registration, preference update, interaction + log, reviews, settings
- Aggregate `GROUP BY` analytics queries

Servlets & sessions

- 10 servlets: GET / POST / PUT / PATCH / DELETE with JSON and proper status codes
- `HttpSession`: new session id at login (fixation defence), 30-minute timeout, HttpOnly cookie
- **Filters:** `AuthFilter` (login + admin role) and security headers (CSP, nosniff)
- Front-end talks to the servlets with `fetch()`

Security PBKDF2 hashing with salt · constant-time compare · login lock after 5 failures · server-side validation · XSS-safe UI (no innerHTML with data)

CODE QUALITY & TESTING

28 automated tests, all passing

TEST CLASS	TESTS	WHAT IT PROVES
UtilTest	6	password hashing, validation, outcome rules
EngineTest	10	each scorer, exclusions, diversity, weights, cold start, decay
IntegrationTest	12	real JDBC database: login lock, registration, **rollback**, preferences, reviews, analytics, admin safeguards

Quality practices

- Package-by-layer: config · model · dao · service · engine · web · util
- One responsibility per class, Javadoc on key classes
- Input validated in one place
- Consistent JSON error contract
- Reproducible demo data (fixed random seed)
- Build: Maven; run: `run.bat`

TEAMWORK & COLLABORATION

Who did what

Vedang Goswami

Team Leader

- Planned the architecture and divided the work
- Built the **recommendation engine**: three scorers, blending, explanations, parallel threads
- Implemented **servlets, filters, sessions** and security
- Merged everyone's work, final testing, this presentation

Ankur Kushwaha

Member

- Designed the **database schema** and ER diagram
- Wrote the **JDBC DAOs** and transaction helper
- Loaded the 251-product catalogue; analytics and reviews services
- Wrote the **JUnit integration tests**

Nitin Raghuvanshi

Member

- Designed the **black-white-yellow UI** (60 / 30 / 10)
- Built Discover, product dialog, admin dashboards
- Researched products and gathered pictures with credits
- README, design diagrams and documentation

How we worked: feature branches, pull requests with one reviewer, small commits, and a green test run before every merge.

Together: weekly review of the rubric checklist and live demo rehearsal.

INNOVATION & EXTRA EFFORT

Beyond the minimum requirements

Explainable AI

Every card says why it was suggested and shows the mix of signals.

Interest Studio

One screen: tell the AI your interests on top, live picks below. Presets, topics and a taste-DNA bar.

Self-tuning advice

Analytics generate suggestions the admin applies in one click.

Real catalogue

251 real items - 162 products, 45 books, 44 films - each with a real picture.

Reviews feed the AI

Star reviews are also behavioural signals for the recommender.

Safe & accessible

Login lock, admin safeguards, keyboard-accessible, responsive layout.

Tech stack: Java 17 · Jakarta Servlets on embedded Jetty · JDBC · H2 (MySQL-compatible SQL) · Gson · JUnit 5 · Maven · HTML / CSS / JavaScript (no framework)

CONCLUSION

A complete, explainable Java recommendation system

What we delivered

- Working Java web application with Servlets, JDBC and sessions
- Hybrid AI recommender with explanations
- Admin settings, analytics and user management
- User preferences, personalised feed and reviews
- 28 passing tests, diagrams and full documentation

Future scope

- Switch to MySQL / PostgreSQL for production
- Learn weights automatically from feedback (A/B tests)
- Embeddings for text and image similarity
- Email / push "picked for you" digests
- Real checkout and inventory integration

Thank you · Questions?

Vedang Goswami (Team Leader) · Ankur Kushwaha · Nitin Raghuvanshi